

Low-Level Design (LLD) Document

How it Works and Why

1. Project Overview

We built a simulation of people editing a shared document at the same time. It uses POSIX message queues to let the different parts talk to each other. There's a server program (it keeps track of the document) and several client programs (they're the ones making changes). A Python script puts everything together: it compiles the code, makes test inputs, starts the programs, grabs the logs, and makes sure everything's working right.

2. What's What

server.c: This is the main brain. It manages a grid of words, like a 10x10 document. It also makes sure only one person changes a word at a time.

client.c: This is what the users use. They can tell it to READ, WRITE, or PRINT the document. It saves the document to output_client<n>.txt regularly.

runner.py: This script gets everything going. This compiles the C code , starts the server and clients , gives them instructions, collects the results and checks if everything went as per plan.

3. How this is Put Together

We have a single server and many clients. They talk using the POSIX message queues. The server has a queue that is named `/osdoc_server``, and each client has its own queue like `/osdoc_client<id>``. Clients send requests to the server's queue (READ, WRITE, etc.). The server sends</id> replies back to the client's own queue.

4. Data and Messages

The messages between the server and clients all have the same basic structure: the type of message, the client's ID, the line and word position, the word itself, how long to lock it, and if the action was successful.

The server keeps track of the document in a grid. Each cell in the grid has the word, whether it's locked, who locked it, and a lock (mutex) to prevent multiple people from changing it at once.

5. Locking Stuff Down

Each word in the document has its own lock (mutex). When someone wants to WRITE, the server grabs the lock for that

word, checks if it's already locked, then locks it for the duration specified. After that time, it unlocks it. READ and PRINT also check the lock status using the same lock. This has a few benefits:

- * Everyone can work on separate words at the same time.
- * We know who locked the word (good for debugging).
- * Words unlock themselves after a certain time.

The client also locks its own message queue to avoid problems when both the main part of the program and the print part try to use the queue at the same time.

6. Grabbing a Snapshot of the Document

Clients periodically ask for a copy of the current document. To do this, they go through each cell in the grid and send a request to the server. To make sure the snapshot is consistent, the client locks its own mutex during the whole process. The server sends back the word in the cell, or `???` if it's locked. The client puts the words together into lines and saves it to a file. If any of the words were locked, the client saves a copy to handle tests that expect `???` in the output.

7. What if Something Goes Wrong?

Message queue operations have timeouts, so the programs don't get stuck forever waiting for a response. When the client starts up, it tries to connect to the server queue several times, in case the server isn't quite ready. The server does the same when connecting to client queues. The runner script also waits a bit between starting programs to avoid problems.

8. Keeping Track of Things

Both the server and the client add timestamps to every log message. The runner script takes these logs and removes the timestamps and puts them in order and then checks if everything happened as it was expected.

9. Why Did We Do It This Way?

POSIX message queues: These are a simple way for different programs to talk to each other.

Per-word locks: This lets the people work on different parts of the document at the same time.

Client-side snapshot lock: This makes sure that the snapshots are consistent even if it slows things down a bit on the client.

10. Error Handling

The server keeps going even if it can not connect to a client queue. If message queue operations fails, the client retries or gives up. The client aborts if it can't connect to the server's message queue. The runner script cleans up any leftover message queues and output files before running tests.

11. How We Test It

Tests are written in JSON files. The runner script compiles the code, creates input files, starts the server and clients, collects the logs, and then checks the logs and output files to make sure everything worked correctly.